

Cellula Week 6 Project 3.1

Evaluator–Generator RAG Platform — Project Report

1. Overview

We built a self-evaluating question-answering platform. A question is first answered by a Generator agent using only the ingested external knowledge, then graded by an independent Evaluator agent. If the answer is not good enough, the Evaluator sends back structured feedback and the Generator tries again, up to a hard cap of 4 iterations. This loop is meant to catch hallucinations and weak answers before the user ever sees them.

The work was split into two clear halves so that both team members owned a full, meaningful vertical slice of the system instead of splitting by task type. Loay owned the knowledge/retrieval/generation side; Adham owned the evaluation/feedback side. Below is what each person built and how the two sides connect.

2. System Flow

End-to-end, a request moves through the platform like this:

- **Ingest:** a source (PDF, DOCX, TXT, code, PPTX, URL, Wikipedia, or WAV) is loaded, its text extracted, chunked, embedded, and stored in the Chroma vector store. Successful ingestion bumps the knowledge-base version.
- **Retrieve:** an incoming question first checks the Redis retrieval cache; on a miss, the vector store returns the top-K most relevant chunks.
- **Generate:** the Generator answers strictly from the retrieved context. If the context doesn't cover the question, it explicitly replies that the information is not available, instead of guessing.
- **Evaluate:** the Evaluator scores the answer on accuracy, relevance, completeness, consistency, grounding, and unsupported claims, then returns ACCEPT or IMPROVE with structured feedback.
- **Loop:** on IMPROVE, feedback goes back to the Generator, which produces a revised answer that is evaluated again. The loop stops at ACCEPT or after 4 iterations, whichever comes first.
- **Cache:** the final answer is cached under a key built from the question, the retrieved context, and the knowledge-base version, so new ingested knowledge automatically invalidates stale cached answers.

The Generator and Evaluator each keep their own memory (GeneratorMemory and EvaluatorMemory). These are deliberately separate instances — the system will not even start if they point to the same object — so neither agent can see or influence the other's internal history directly.

3. Loay — Knowledge, Retrieval & Generation

Loay owned everything responsible for getting knowledge into the system and turning retrieved context into a grounded answer.

3.1 Ingestion pipeline

Built the full pipeline: Input → Loader → Content extraction → Text normalization → Chunking → Embeddings → Vector store → Retriever. Supported source types:

- PDF, DOCX, TXT
- Source code files
- PPT / PPTX
- Web URLs and Wikipedia pages
- WAV audio, transcribed to text via Whisper

3.2 Retrieval

- Embedding generation for ingested chunks
- The Chroma vector store and document metadata
- The retriever: similarity search and top-K context formatting
- Source references / citations exposed alongside answers

3.3 Generator (LLM)

Implemented the Generator agent that takes the retrieved context and the question and produces an answer strictly grounded in that context. If the retrieved context doesn't actually cover the question, the Generator returns an explicit “information not available” response instead of hallucinating.

3.4 Generator Memory

Implemented GeneratorMemory, isolated from the Evaluator, holding:

- Conversation context
- Previous answer attempts
- Retrieved context used for each attempt
- Improvement attempts made in response to Evaluator feedback

3.5 Generator-side Redis caching

Owned caching for the ingestion/retrieval half of the system:

- Cached processed documents so re-ingesting the same file is instant
- Cached embeddings
- Cached retrieval results

4. Adham — Evaluation & Feedback

Adham owned the entire evaluation layer: judging answers, generating feedback, and closing the loop back to the Generator.

4.1 Evaluator (LLM)

Built the Evaluator agent. Input: the original question, the current answer, and relevant evaluation context. Output: a structured decision — ACCEPT or IMPROVE — with a score and feedback.

4.2 Evaluation criteria

Implemented scoring logic covering all the criteria required by the spec:

- Accuracy
- Relevance
- Completeness
- Consistency with external knowledge
- Grounding
- Unsupported claims
- Overall quality

4.3 Evaluator Memory

Implemented EvaluatorMemory, a separate instance from GeneratorMemory, holding:

- Evaluation history
- Previous feedback

- Previous decisions
- Evaluation criteria / rules

4.4 Feedback mechanism

Designed the feedback representation that flows back to the Generator on an IMPROVE decision — concrete enough that the Generator can act on it and actually produce a better next answer, rather than a generic “try again.”

4.5 Evaluator-side Redis caching

Owned caching for the evaluation half of the system:

- Cached evaluation results
- Cached repeated evaluation requests
- Cached LLM responses where appropriate

5. Ownership Summary

Loay

Component	Details
Ingestion	PDF, DOCX, TXT, code, PPTX, URL, Wikipedia, WAV → Whisper
Retrieval	Embeddings, Chroma vector store, retriever, top-K context, citations
Generator LLM	Grounded answer generation + “not available” fallback
Generator Memory	Context, answer attempts, retrieved context, improvement attempts
Redis (Generator side)	Documents, embeddings, retrieval results

Adham

Component	Details
Evaluator LLM	ACCEPT / IMPROVE decision, score, feedback
Evaluation criteria	Accuracy, relevance, completeness, consistency, grounding, unsupported claims, quality
Evaluator Memory	Evaluation history, feedback, decisions, criteria/rules
Feedback mechanism	Structured feedback routed back to the Generator
Redis (Evaluator side)	Evaluation results, repeated requests, LLM responses

6. How It All Connects

The two sides meet at one clean interface: Loay's Generator produces an answer from retrieved knowledge, and Adham's Evaluator consumes that answer and decides ACCEPT or IMPROVE. On IMPROVE, Adham's feedback flows back into Loay's Generator, which produces a new attempt using the same retrieved context. This repeats until ACCEPT or the 4-iteration cap is reached, at which point the latest answer is returned, flagged as not fully validated if the cap was hit. The orchestrator (shared) owns the loop counter and enforces that GeneratorMemory and EvaluatorMemory always stay separate instances, so each person's memory implementation is used strictly for its own agent.